Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Системы искусственного интеллекта

Лабораторная работа №4

Метод k ближайших соседей (kNN)

Выполнил
Чэнь Хаолинь P3316 407960

Преподаватель
Болдырева Елена Александровна

г. Санкт-Петербург, 2026

Содержание

1 Цель работы и теоретические сведения **2**

2 Разбиение и предварительная обработка **2**

3 Описательная статистика **3**

3.1 Wine 3

3.2 Diabetes 4

4 Реализация алгоритма и метрик **5**

4.1 Класс KNN 5

4.2 Метрики качества 6

5 Подбор гиперпараметров **7**

5.1 Классификация Wine 7

5.2 Регрессия Diabetes 8

6 Итоговая оценка на тестовой выборке **9**

7 Заключение **9**

1. Цель работы и теоретические сведения

Цель работы — реализовать метод k ближайших соседей для классификации вин и регрессии показателя прогрессирования диабета, сравнить число соседей и метрики расстояния на валидационной выборке, затем оценить выбранные модели на тестовой выборке.

При обучении kNN сохраняет признаки и ответы обучающих объектов. Для каждого нового объекта алгоритм вычисляет расстояния до обучающих точек, упорядочивает их и выбирает первые k соседей. При классификации прогнозом служит класс с наибольшим числом голосов. При регрессии прогноз равен среднему ответу $\hat{y} = k^{-1} \sum_{i \in \mathcal{N}_k(x)} y_i$, где $\mathcal{N}_k(x)$ обозначает множество выбранных соседей.

Для векторов признаков $x, z \in \mathbb{R}^p$ использованы три расстояния:

$$
\begin{aligned}
d_2(x, z) &= \sqrt{\sum_{j=1}^{p} (x_j - z_j)^2} && \text{(евклидово)},\\
d_1(x, z) &= \sum_{j=1}^{p} |x_j - z_j| && \text{(манхэттенское)},\\
d_\infty(x, z) &= \max_{1 \le j \le p} |x_j - z_j| && \text{(Чебышёва)}.
\end{aligned}
$$

В L_2 крупные отклонения получают больший вклад из-за возведения в квадрат. В L_1 абсолютные отклонения суммируются, а L_∞ учитывает только максимальное отклонение. Увеличение k обычно сглаживает прогнозы, снижая их чувствительность к отдельным наблюдениям.

2. Разбиение и предварительная обработка

На первом шаге выделены около 40 % объектов с `random_state=42`. Эта часть поровну разбита на валидацию и тест с `random_state=43`. Для Wine оба разбиения стратифицированы по целевому классу, для Diabetes использовано случайное разбиение. Размеры выборок приведены в таблице ниже. Доли близки к 60/20/20 % с учётом целого числа объектов.

Таблица 1: Размеры выборок и распределение классов Wine

Набор	Всего	Обучение	Валидация	Тест
Wine	178	106	36	36
Класс 0	59	35	12	12
Класс 1	71	42	14	15
Класс 2	48	29	10	9
Diabetes	442	265	88	89

Оба набора содержат конечные значения во всех столбцах. Каждый признак стандартизован по формуле $z_j = (x_j - \mu_j)/\sigma_j$. Среднее μ_j и стандартное отклонение σ_j с делением на число объектов вычислены по обучающей выборке. Эти же параметры применены к валидации и тесту. При $\sigma_j = 0$ делитель заменяется единицей. Целевые значения сохраняют исходный масштаб.

3. Описательная статистика

Статистика рассчитана для исходных значений по каждому набору целиком. Она используется для описания данных. В таблицах s обозначает выборочное стандартное отклонение с делением на $n-1$, а Q_1 и Q_3 соответствуют 25-му и 75-му процентилям.

3.1. Wine

В наборе 178 объектов, 13 химических признаков и три класса вина. На класс 0 приходится 33,1 % наблюдений, на класс 1 — 39,9 %, на класс 2 — 27,0 %. Различие масштабов видно по признакам `proline` и `hue`, для которых s равно 314,907 и 0,229 соответственно.

Таблица 2: Описательная статистика признаков Wine

Признак	Среднее	s	Мин.	Q_1	Медиана	Q_3	Макс.
`alcohol`	13,001	0,812	11,030	12,362	13,050	13,678	14,830
`malic_acid`	2,336	1,117	0,740	1,603	1,865	3,083	5,800
`ash`	2,367	0,274	1,360	2,210	2,360	2,558	3,230
`alcalinity_of_ash`	19,495	3,340	10,600	17,200	19,500	21,500	30,000
`magnesium`	99,742	14,282	70,000	88,000	98,000	107,000	162,000
`total_phenols`	2,295	0,626	0,980	1,742	2,355	2,800	3,880
`flavanoids`	2,029	0,999	0,340	1,205	2,135	2,875	5,080
`nonflavanoid_phenols`	0,362	0,124	0,130	0,270	0,340	0,438	0,660
`proanthocyanins`	1,591	0,572	0,410	1,250	1,555	1,950	3,580
`color_intensity`	5,058	2,318	1,280	3,220	4,690	6,200	13,000
`hue`	0,957	0,229	0,480	0,782	0,965	1,120	1,710
`od280/od315_of_diluted_wines`	2,612	0,710	1,270	1,938	2,780	3,170	4,000
`proline`	746,893	314,907	278,000	500,500	673,500	985,000	1680,000

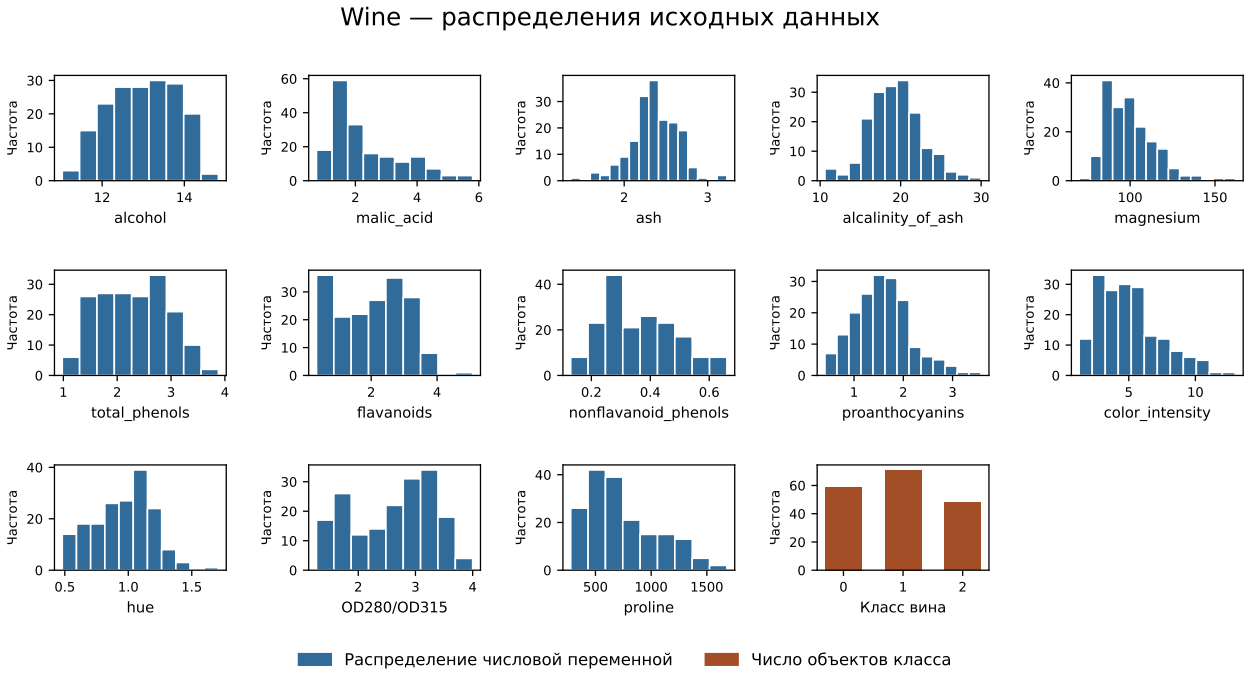


Рис. 1: Распределения признаков и частоты классов Wine

3.2. Diabetes

В наборе 442 наблюдения и 10 исходных признаков, среди которых возраст, индекс массы тела, артериальное давление и лабораторные показатели. Вызов `load_diabetes(scaled=False)` возвращает признаки в исходном масштабе. Переменная `sex` представлена кодами 1 и 2. Целевой показатель характеризует прогрессирование болезни через год и принимает значения от 25 до 346.

Таблица 3: Описательная статистика признаков и целевой переменной Diabetes

Признак	Среднее	s	Мин.	Q_1	Медиана	Q_3	Макс.
`age`	48,518	13,109	19,000	38,250	50,000	59,000	79,000
`sex`	1,468	0,500	1,000	1,000	1,000	2,000	2,000
`bmi`	26,376	4,418	18,000	23,200	25,700	29,275	42,200
`bp`	94,647	13,831	62,000	84,000	93,000	105,000	133,000
`s1`	189,140	34,608	97,000	164,250	186,000	209,750	301,000
`s2`	115,439	30,413	41,600	96,050	113,000	134,500	242,400
`s3`	49,788	12,934	22,000	40,250	48,000	57,750	99,000
`s4`	4,070	1,290	2,000	3,000	4,000	5,000	9,090
`s5`	4,641	0,522	3,258	4,277	4,620	4,997	6,107
`s6`	91,260	11,496	58,000	83,250	91,000	98,000	124,000
`target`	152,133	77,093	25,000	87,000	140,500	211,500	346,000

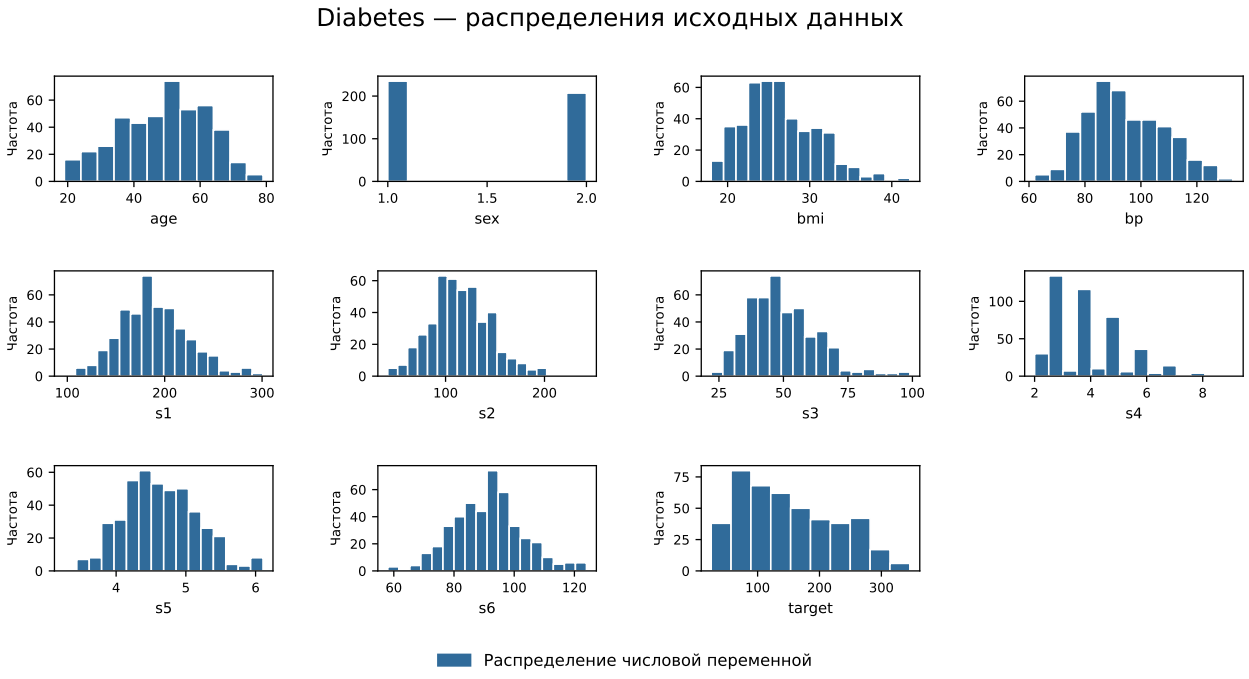


Рис. 2: Распределения признаков и целевого показателя Diabetes

4. Реализация алгоритма и метрик

4.1. Класс KNN

Метод `fit` сохраняет обучающие данные, а `predict` выбирает соседей. При равных расстояниях сохраняется порядок обучающих строк. При равенстве голосов выигрывает класс с наименьшей суммой расстояний до проголосовавших за него соседей, при равенстве сумм — наименьшая метка.

```
"""Brute-force k-nearest neighbors for numeric features and targets."""

import numpy as np


DISTANCES = ("euclidean", "manhattan", "chebyshev")


class KNN:
    def __init__(self, k, task, distance):
        if not isinstance(k, (int, np.integer)) or k < 1:
            raise ValueError("k must be a positive integer.")
        if task not in ("classification", "regression"):
            raise ValueError("Unknown task.")
        if distance not in DISTANCES:
            raise ValueError("Unknown distance.")
        self.k = k
        self.task = task
        self.distance = distance
        self._features = None

    def fit(self, features, targets):
        features = np.asarray(features, dtype=float)
        targets = np.asarray(targets)
        if features.ndim != 2 or features.shape[1] == 0:
            raise ValueError("Expected a nonempty feature matrix.")
        if targets.shape != (len(features),) or len(features) < self.k:
            raise ValueError("Expected one target per row and at least k rows.")
        self._features = features.copy()
        self._targets = targets.copy()
        return self

    def predict(self, features):
        if self._features is None:
            raise ValueError("Fit the model before predicting.")
        features = np.asarray(features, dtype=float)
        if features.ndim != 2 or features.shape[1] != self._features.shape[1]:
            raise ValueError("Unexpected feature matrix shape.")
        dtype = self._targets.dtype if self.task == "classification" else float
        predictions = np.empty(len(features), dtype=dtype)
        for row, query in enumerate(features):
            distances = self._distances(query)
            # Stable sorting keeps training row order at equal distances.
            neighbors = np.argsort(distances, kind="stable")[:self.k]
            targets = self._targets[neighbors]
            if self.task == "classification":
                predictions[row] = self._vote(targets, distances[neighbors])
            else:
                predictions[row] = targets.mean()
        return predictions

    def _distances(self, query):
        difference = np.abs(self._features - query)
        if self.distance == "euclidean":
            return np.sqrt(np.sum(difference ** 2, axis=1))
        if self.distance == "manhattan":
            return difference.sum(axis=1)
        return difference.max(axis=1)

    @staticmethod
    def _vote(targets, distances):
        labels, counts = np.unique(targets, return_counts=True)
        leaders = labels[counts == counts.max()]
        # Among vote leaders, prefer the smallest distance sum, then label.
        return min(leaders, key=lambda c: (distances[targets == c].sum(), c))
```

4.2. Метрики качества

В матрице ошибок строки соответствуют фактическим классам, столбцы — предсказанным. Доля правильных ответов (accuracy) равна сумме диагональных элементов, делённой на объём выборки. Для каждого класса c вычислены

$$P_c = \frac{TP_c}{TP_c + FP_c}, \qquad R_c = \frac{TP_c}{TP_c + FN_c}, \qquad F_{1,c} = \frac{2P_cR_c}{P_c + R_c}.$$

При нулевом знаменателе соответствующая оценка принимается равной нулю. Macro-метрики — арифметические средние P_c, R_c и $F_{1,c}$ по классам. Для регрессии $e_i = y_i - \hat{y}_i$, а $\bar{y}$ — среднее фактическое значение в оцениваемой выборке:

$$\mathrm{MAE} = \frac{1}{n}\sum_i |e_i|, \qquad \mathrm{MSE} = \frac{1}{n}\sum_i e_i^2, \qquad R^2 = 1 - \frac{\sum_i e_i^2}{\sum_i (y_i - \bar{y})^2}.$$

```
"""Classification and regression scores implemented with NumPy."""

import numpy as np


def confusion_matrix(actual, predicted):
    # Sorted labels define both axes. Rows are actual, columns predicted.
    labels = np.union1d(actual, predicted)
    matrix = np.zeros((len(labels), len(labels)), dtype=int)
    rows = np.searchsorted(labels, actual)
    columns = np.searchsorted(labels, predicted)
    np.add.at(matrix, (rows, columns), 1)
    return labels, matrix


def _divide(numerator, denominator):
    # A class score with an undefined denominator contributes zero.
    return np.divide(
        numerator, denominator,
        out=np.zeros_like(numerator, dtype=float), where=denominator != 0,
    )


def classification_scores(matrix):
    tp = matrix.diagonal()
    precision = _divide(tp, matrix.sum(axis=0))
    recall = _divide(tp, matrix.sum(axis=1))
    f1 = _divide(2 * precision * recall, precision + recall)
    return {
        "accuracy": tp.sum() / matrix.sum(),
        "precision_macro": precision.mean(),
        "recall_macro": recall.mean(),
        "f1_macro": f1.mean(),
    }


def regression_scores(actual, predicted):
    residuals = actual - predicted
    squared_errors = residuals ** 2
    sst = np.sum((actual - actual.mean()) ** 2)
    # R² is undefined for a constant target under the stated formula.
    return {
        "mae": np.abs(residuals).mean(),
        "mse": squared_errors.mean(),
        "r2": 1 - squared_errors.sum() / sst if sst else np.nan,
    }
```

5. Подбор гиперпараметров

Для каждой задачи проверены 12 комбинаций $k \in \{3, 5, 10, 15\}$ и трёх расстояний. Для Wine выбран максимум macro-F1, для Diabetes — минимум MSE. При равенстве целевой метрики приоритет получает меньшее k, затем порядок L_2, L_1, L_∞. Жирным шрифтом выделена выбранная строка.

5.1. Классификация Wine

Таблица 4: Метрики классификации на валидационной выборке

Расстояние	k	Accuracy	Macro-precision	Macro-recall	Macro-F1
L_2	3	0,9722	0,9744	0,9762	0,9743
L_2	5	0,9722	0,9744	0,9762	0,9743
L_2	10	1,0000	1,0000	1,0000	1,0000
L_2	15	1,0000	1,0000	1,0000	1,0000
L_1	3	0,9722	0,9744	0,9762	0,9743
L_1	**5**	**1,0000**	**1,0000**	**1,0000**	**1,0000**
L_1	10	0,9722	0,9744	0,9762	0,9743
L_1	15	0,9444	0,9524	0,9524	0,9487
L_∞	3	0,9167	0,9154	0,9190	0,9163
L_∞	5	0,8889	0,8897	0,8857	0,8864
L_∞	10	0,8611	0,8685	0,8579	0,8621
L_∞	15	0,8889	0,8913	0,8913	0,8913

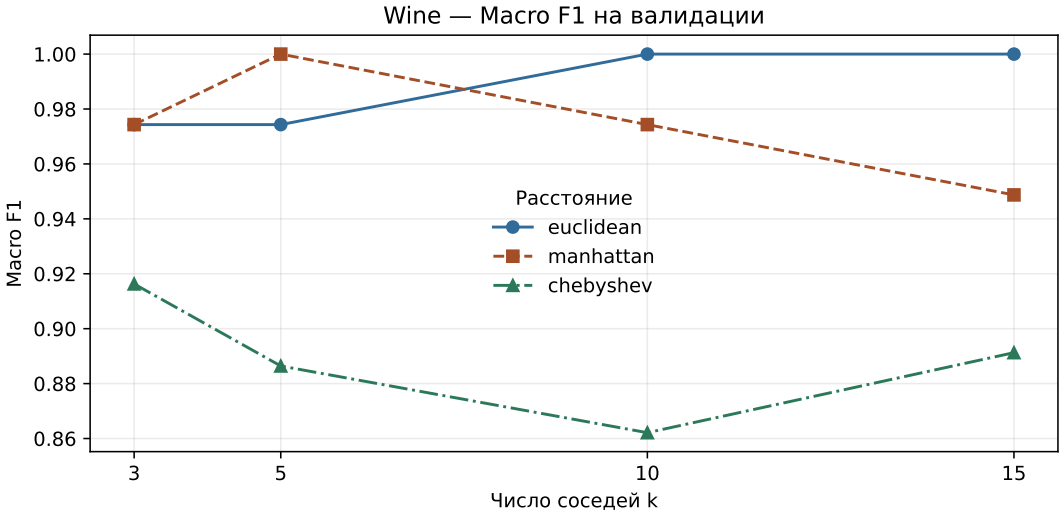


Рис. 3: Зависимость macro-F1 от числа соседей для Wine

При L_2 значение macro-F1 выросло с 0,9743 при $k = 3, 5$ до 1,0000 при $k = 10, 15$. Манхэттенское расстояние дало тот же максимум при $k = 5$, поэтому выбрана эта конфигурация. При L_1 дальнейшее увеличение k ухудшило оценку до 0,9487 при $k = 15$. Расстояние Чебышёва уступило двум другим при всех четырёх значениях k.

5.2. Регрессия Diabetes

Таблица 5: Метрики регрессии на валидационной выборке

Расстояние	k	MAE	MSE	R^2
L_2	3	42,46	2849,78	0,5043
L_2	5	44,58	3090,42	0,4624
L_2	10	45,28	2931,29	0,4901
L_2	15	43,95	2880,22	0,4990
L_1	3	45,15	3130,57	0,4554
L_1	5	45,39	3087,63	0,4629
L_1	10	44,80	2995,88	0,4788
L_1	15	46,64	3165,15	0,4494
L_∞	3	48,74	3625,56	0,3693
L_∞	5	46,19	3064,54	0,4669
L_∞	10	45,30	2867,53	0,5012
L_∞	**15**	**45,24**	**2784,04**	**0,5157**

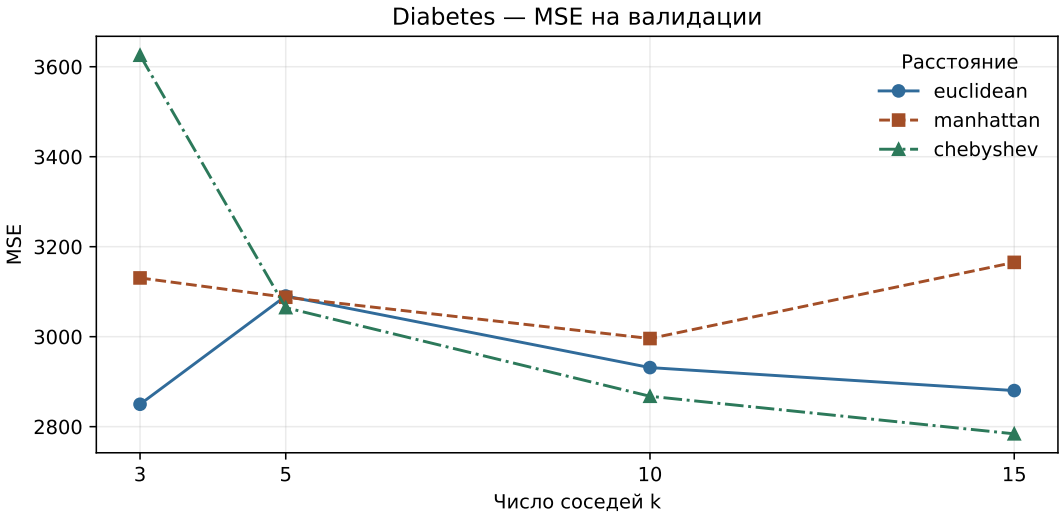


Рис. 4: Зависимость MSE от числа соседей для Diabetes

Для L_∞ MSE последовательно снизилась с 3625,56 при $k = 3$ до 2784,04 при $k = 15$. Евклидово расстояние дало свой минимум 2849,78 при $k = 3$, а манхэттенское — 2995,88 при $k = 10$. Евклидова модель с $k = 3$ также дала наименьшую MAE (42,46). По принятому критерию MSE выбрана модель с расстоянием Чебышёва и $k = 15$.

6. Итоговая оценка на тестовой выборке

На тесте оценена одна конфигурация для каждой задачи. Обучающая выборка и параметры стандартизации остались теми же, что при валидации.

Таблица 6: Метрики выбранной модели Wine, $k = 5$, L_1, $n = 36$

Accuracy	Macro-precision	Macro-recall	Macro-F1
0,9444	0,9410	0,9556	0,9453

Таблица 7: Метрики выбранной модели Diabetes, $k = 15$, L_∞, $n = 89$

MAE	MSE	R^2
45,90	3305,46	0,4278

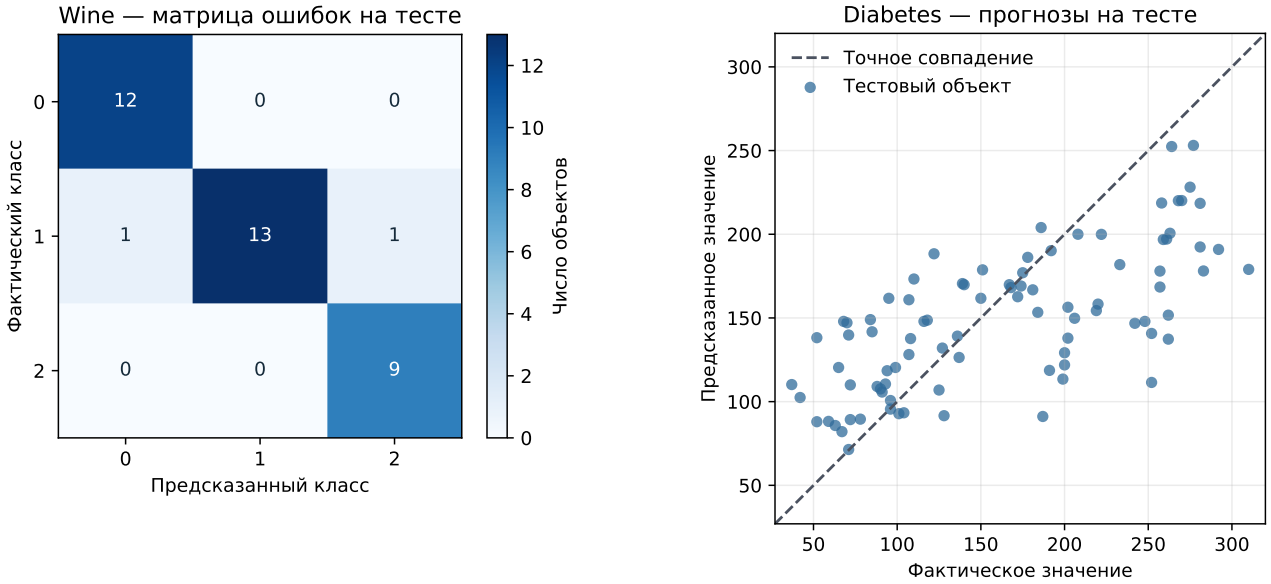


Рис. 5: Матрица ошибок Wine и сравнение фактических значений с прогнозами Diabetes

Модель Wine верно классифицировала 34 из 36 объектов. Обе ошибки допущены для объектов класса 1. Один объект отнесён к классу 0, другой — к классу 2.

Для Diabetes положительное R^2 означает, что сумма квадратов ошибок модели ниже, чем при прогнозировании средним фактическим значением теста. Прогнозы лежат в диапазоне от 71,4 до 253,1, а фактические ответы — от 37 до 310. При больших фактических значениях часть точек заметно ниже диагонали. Такая картина согласуется со сглаживанием прогноза при усреднении по 15 соседям.

7. Заключение

На выбранном разбиении Wine модель с L_1 и $k = 5$ получила на тесте macro-F1 0,9453. Для Diabetes модель с L_∞ и $k = 15$ дала MSE 3305,46 и $R^2 = 0{,}4278$. В классификации увеличение k улучшило macro-F1 для L_2, тогда как для L_1 максимум достигнут при промежуточном k. В регрессии монотонное снижение MSE с ростом k наблюдалось для L_∞. Эти результаты отражают совместное влияние числа соседей и расстояния при фиксированном разбиении данных.